

embedded-xml-schema

A memory-safe, validating XML parser for embedded Rust

Project Brief · Requirements · Software Specification · Architecture · Design Patterns · Test Architecture

Recommended repository: embedded-xml-schema
Alternative compatibility-oriented name: miniml-parser-rs

Version 0.1 - Architecture baseline
5 October 2026

Document control

Item	Value
Purpose	Implementation-ready baseline for a new Rust project inspired by miniML-Parser.
Primary language	English (requirements and specification).
Target	Embedded / systems environments; host tools for schema generation.
Core safety promise	Parser core is <code>#![no_std]</code> and <code>#![forbid(unsafe_code)]</code> .
Coverage policy	Unit tests alone must reach 100% coverage of production logic; integration and N2N/E2E are additional independent layers.
Status	Proposed baseline. Architectural decisions are intentionally explicit so implementation can begin immediately.

Naming recommendation

Use "embedded-xml-schema" for the repository and public crate if available. It describes the real capability instead of tying the project permanently to the original C implementation. Keep "miniML-Parser compatible" as a compatibility goal, not as the product identity.

Contents

1. Project / User Brief
2. Requirements (English)
3. Software Specification
4. Architecture
5. Design Patterns and Architectural Patterns
6. Test Architecture
7. Repository and Module Structure
8. CI/CD and Quality Gates
9. Implementation Roadmap
10. Risks, Constraints and Security Considerations
11. Definition of Done / Acceptance Checklist
12. Source Baseline and References

1. Project / User Brief

1.1 Background

The project creates a small, validating XML parser in Rust for embedded and systems software. It is inspired by the design goals of the C project miniML-Parser: a compact parser, schema-driven validation, extraction of XML content into typed targets, and a host-side generator that converts an XML Schema definition into code usable by the parser. The Rust project is not intended to be a line-by-line translation. Its goal is to preserve the useful embedded behavior while redesigning ownership, safety, APIs and testing around Rust.

The original project exposes a single top-level parse call, validates against a schema tree, extracts typed values, supports callbacks and several target-addressing modes, and includes a code generator for schema-derived C structures. Its README reports a code footprint of about 1.8 kB for the parser in one embedded compiler configuration. Those capabilities form the behavioral baseline, while this specification deliberately adds stronger safety, deterministic resource controls, richer diagnostics and first-class Rust ergonomics.

Core product idea

A tiny schema-aware XML parser that can be used safely in firmware and other constrained software without a heap, while still offering generated typed models and an optional C-compatible migration path.

1.2 Problem statement

Embedded systems frequently need to consume small configuration, provisioning, command or telemetry documents. Full XML stacks are often too large or require allocation, while hand-written parsers and small C libraries can expose memory-safety risks and require manual schema-to-structure mapping. The project must provide a small and deterministic middle ground: validation plus typed extraction without bringing a large runtime or unsafe parser core.

1.3 Primary users

- Firmware and embedded Rust developers who need deterministic XML parsing without std.
- C/C++ embedded teams that want a gradual migration path through an optional C ABI.
- Systems developers who need generated, typed structures from a constrained XSD subset.
- Library maintainers who require strong malformed-input resistance and reproducible resource bounds.

1.4 Value proposition

Value	Meaning
Memory safety	All parser logic is safe Rust. Unsafe code is forbidden in the core crate.
Embedded suitability	no_std first; heap allocation is optional rather than assumed.
Deterministic behavior	Explicit input, depth, token and occurrence limits prevent uncontrolled resource growth.
Typed extraction	Schema-driven conversion produces application values without ad-hoc string handling.
Migration path	Optional C ABI can replace or coexist with legacy C parsing at a narrow boundary.
Tooling	Host-side XSD subset compiler generates Rust and, optionally, C compatibility models.
Verifiability	100% unit coverage is a hard gate before integration and N2N/E2E testing.

1.5 Success criteria

- A useful XML + schema subset can be parsed and validated with no heap allocation.
- The core compiles with `#![no_std]` and `#![forbid(unsafe_code)]`.
- All production logic reaches 100% coverage using unit tests alone.
- Malformed XML never causes panic, undefined behavior, out-of-bounds access or unbounded recursion.
- Generated Rust types and schema descriptors compile without manual edits for supported XSD inputs.
- N2N/E2E proves XSD -> generated code -> build -> XML parse -> typed output.
- Optional C consumers can invoke the Rust implementation through a deliberately small ABI.

1.6 Non-goals for v1

- Full W3C XML 1.0/1.1 and full XSD 1.0/1.1 implementation.
- DOM construction as the default model.
- XPath, XQuery, XSLT, DTD validation, external entity loading or network access.
- Transparent support for arbitrary encodings; UTF-8 is the required v1 encoding.
- A general-purpose desktop XML framework competing with quick-xml, roxmltree or libxml2.

2. Requirements (English)

The following requirements are normative for the initial implementation unless an Architecture Decision Record (ADR) explicitly supersedes them.

2.1 Functional requirements

ID	Requirement
FR-001	Parse complete XML documents from an immutable UTF-8 byte/string input.
FR-002	Validate element names, hierarchy, child order and occurrence bounds against a compiled schema model.
FR-003	Validate declared attributes, required/optional status and supported attribute value types.
FR-004	Extract element text and attribute values into typed Rust values.
FR-005	Support at minimum string, boolean, signed integer, unsigned integer and floating-point content types in v1.
FR-006	Support numeric minimum/maximum facets and string length constraints in the schema model.
FR-007	Support minOccurs/maxOccurs equivalents within an explicitly bounded implementation range.
FR-008	Return structured errors containing category, reason and source position where determinable.
FR-009	Allow application-defined element completion hooks without requiring heap allocation.
FR-010	Provide a schema code generator that accepts the supported XSD subset and emits Rust types plus static schema descriptors.

ID	Requirement
FR-011	Optionally emit C-compatible structures/descriptors for legacy interoperability.
FR-012	Provide an optional C ABI crate exposing a stable, minimal parsing surface.
FR-013	Support caller-provided resource limits for document bytes, nesting depth, attributes, children and occurrences.
FR-014	Provide deterministic rejection for XML features intentionally unsupported in v1.
FR-015	Expose parser behavior through a small safe Rust facade with sensible defaults and an explicit configuration builder.
FR-016	Permit no-allocation operation for schemas and target types that do not require dynamic storage.
FR-017	Permit optional alloc-backed convenience targets behind a feature flag.
FR-018	Preserve borrowed slices where possible rather than copying XML content.
FR-019	Validate the entire document; successful return means syntax, schema and extraction all succeeded.
FR-020	Provide machine-readable compatibility manifests for generated code to detect generator/runtime version mismatches.

2.2 Non-functional requirements

ID	Requirement
NFR-001	The parser core SHALL compile with <code>#![no_std]</code> .
NFR-002	The parser core SHALL use <code>#![forbid(unsafe_code)]</code> .
NFR-003	The core SHALL NOT perform I/O, filesystem access, network access or global mutable state.
NFR-004	Parsing SHALL be deterministic for identical input, schema and configuration.
NFR-005	The default parsing algorithm SHALL be non-recursive or shall enforce a strict configured depth bound before recursion can exhaust the stack.
NFR-006	The no-alloc profile SHALL not allocate dynamically.
NFR-007	Panics SHALL NOT be used for expected malformed-input control flow.
NFR-008	Public APIs SHALL document time, memory and lifetime behavior.

ID	Requirement
NFR-009	Unit tests alone SHALL produce 100% line and function coverage for production logic; region/branch coverage SHALL target 100% where tooling can measure it reliably.
NFR-010	Integration and N2N/E2E suites SHALL be additional quality gates and SHALL NOT be required to raise unit coverage to 100%.
NFR-011	Fuzzing SHALL include malformed, truncated, deeply nested and adversarial documents.
NFR-012	The project SHALL support reproducible builds and a pinned minimum supported Rust version (MSRV).
NFR-013	Core dependencies SHALL be minimized and every runtime dependency SHALL be justified in an ADR.
NFR-014	Generated code SHALL be deterministic: the same schema and generator version produce byte-for-byte stable output except explicitly documented metadata.
NFR-015	The public Rust API SHALL follow semantic versioning; C ABI compatibility policy SHALL be documented separately.

2.3 Security requirements

ID	Requirement
SEC-001	External entities, external DTDs and network-loaded entities are forbidden.
SEC-002	Entity expansion beyond the explicitly supported predefined XML entities is forbidden in v1.
SEC-003	All index arithmetic SHALL use checked/saturating logic where overflow could affect bounds.
SEC-004	Every parser loop SHALL make forward progress or return an error.
SEC-005	Maximum nesting depth and document size SHALL be enforced before resource exhaustion.
SEC-006	Conversion overflow and underflow SHALL return typed errors rather than clamp silently.
SEC-007	FFI pointers SHALL be validated at the boundary to the extent possible; raw pointers SHALL NOT enter the core API.
SEC-008	The C ABI crate SHALL contain all project unsafe code unless an ADR explicitly creates another isolated unsafe boundary.
SEC-009	Fuzz regressions SHALL be converted into permanent unit tests before closure.
SEC-010	No parser behavior SHALL depend on locale or ambient process state.

2.4 Compatibility requirements

ID	Requirement
COMP-001	Behavioral compatibility with miniML-Parser concepts is desirable where it does not conflict with Rust safety or the defined v1 subset.
COMP-002	The Rust-native API is primary; source-level emulation of the C API is secondary.
COMP-003	The optional C ABI SHALL expose opaque handles or POD-compatible descriptors rather than Rust layout-dependent types.
COMP-004	Generated C artifacts SHALL use fixed-width integer types and explicit ownership rules.
COMP-005	Cross-language N2N/E2E tests SHALL compile and run at least one C consumer against the Rust static library on CI host targets.

3. Software Specification

3.1 Supported XML profile (v1)

Feature	v1 policy
Encoding	UTF-8 only.
Elements	Supported; properly nested, case-sensitive names.
Attributes	Supported; quoted values; duplicate attributes rejected.
Text content	Supported; borrowed when possible.
CDATA	Optional feature; if enabled, exposed as text content.
Comments	Recognized and skipped; not materialized.
Processing instructions	Recognized and skipped, or rejected under strict mode.
Namespaces	Phase 2. v1 may treat colon as a legal name character without namespace resolution.
DTD / external entities	Rejected.
General entity declarations	Rejected.
Predefined entities	amp, lt, gt, apos, quot supported.
Character references	Decimal and hexadecimal numeric references supported with Unicode validity checks.
Mixed content	Not part of the initial typed-schema profile; reject unless schema explicitly opts in later.

3.2 Supported schema profile

The generator accepts a deliberately constrained XSD subset and lowers it to a canonical Schema IR. The runtime parser never interprets XSD text; it receives generated static descriptors. This keeps host-side complexity out of embedded targets.

Schema capability	v1
Named elements	Yes
Nested complex structures	Yes, deterministic child ordering
minOccurs / maxOccurs	Yes, bounded
Required / optional attributes	Yes
Primitive simple types	string, bool, signed/unsigned integers, float
Numeric min/max facets	Yes
String length facets	Yes
Enumerations	Recommended for v1
Choice	Phase 2 unless simple deterministic alternatives can be generated
Namespaces/import/include	Phase 2 / host-side only
Pattern facets / regex	Phase 2
Mixed content	No
any / anyAttribute	No
Substitution groups	No
Identity constraints	No

3.3 Public Rust API

Illustrative API - final names may change

```

#![no_std]

use embedded_xml_schema::{Parser, ParserConfig, Schema, ParseError};

let config = ParserConfig::builder()
    .max_depth(16)
    .max_attributes_per_element(8)
    .max_document_bytes(4096)
    .build();

let parser = Parser::new(&MY_SCHEMA, config);
let value: DeviceConfig<'_> = parser.parse(xml_bytes)?;
```

The preferred native API owns no global state. Parser is a lightweight value containing references to immutable schema data and configuration. The returned target may borrow from the input, allowing zero-copy string fields where the generated model permits it.

3.4 Core abstractions

Abstraction	Responsibility
Parser	Facade coordinating tokenization, state transitions, validation and extraction.
ParserConfig	Immutable resource and strictness configuration created via Builder.
Schema / SchemaNode	Static, generated description of allowed elements, attributes, types, cardinality and facets.
Tokenizer / Cursor	Bounds-safe, forward-only view over input with position tracking.
ParserState	Explicit deterministic state machine for XML syntax.
Validator	Applies structural and value Specifications to parser events.
TargetSink	Strategy trait receiving validated values and building typed output.
ParseError	Structured error with category, code and position.
ResourceBudget	Tracks configured hard limits.
Hook	Optional event/callback interface invoked at safe lifecycle points.

3.5 Error model

Category	Examples
Syntax	Unexpected EOF, malformed tag, unclosed quote, invalid character reference.
Schema	Unexpected element, wrong order, missing required child, occurrence limit exceeded.
Attribute	Unknown attribute, duplicate attribute, missing required attribute.
Conversion	Invalid boolean, integer overflow, float format error.
Facet	Below minimum, above maximum, invalid length, invalid enumeration value.
Resource	Document too large, depth exceeded, token budget exceeded.
Unsupported	DTD, external entity, unsupported XSD-generated feature.
Internal contract	Generated/runtime version mismatch. Must not represent malformed-input panic paths.

Suggested diagnostic shape

```
pub struct ParseError {
    pub kind: ErrorKind,
    pub position: Position,
    pub schema_node: Option<SchemaNodeId>,
}

pub struct Position {
    pub byte_offset: u32,
    pub line: u32,
    pub column: u32,
}
```

3.6 Feature flags

Feature	Default	Purpose
alloc	off	Owned strings/vectors and dynamic convenience targets.
std	off	std::error::Error, filesystem helpers in facade only; implies alloc.
cdata	off or on by decision	CDATA token support.
callbacks	off	Application lifecycle hooks.
ffi	separate crate	C ABI; never a core feature.
serde	off	Optional host-side convenience only, not required by parser core.

3.7 Resource model

The parser must make memory and CPU behavior explicit. Configuration limits are checked before expensive work. The no-alloc profile stores parser state on the stack and references static schema data. Repetition targets must use fixed-capacity arrays, caller-owned buffers, callbacks/sinks or an alloc-enabled profile.

Budget	Purpose
max_document_bytes	Reject oversized documents immediately.
max_depth	Prevent stack/state exhaustion through deep nesting.
max_attributes_per_element	Bound attribute scanning work.
max_children_per_element	Bound schema/event bookkeeping.
max_text_bytes	Bound conversions and target buffers.
max_occurrences	Bound repeated schema elements even if XSD says unbounded.
max_tokens	Optional global work budget for adversarial input.

4. Architecture

4.1 Architectural principles

- Safe core, unsafe edge: the parser core contains no unsafe code; FFI is physically separated.
- Dependency inversion: parsing logic depends on traits and immutable schema contracts, not on generated application types.
- Host/target separation: XSD parsing and code generation run on the host; embedded targets receive static generated descriptors.
- Zero-copy by default: borrow input slices unless decoding or type conversion requires materialization.
- Fail closed: unsupported XML/XSD features are rejected explicitly.
- Deterministic resources: resource limits are part of the public contract, not hidden implementation details.
- Testability by construction: state transitions and validators are pure or near-pure components with direct unit entry points.

4.2 System context

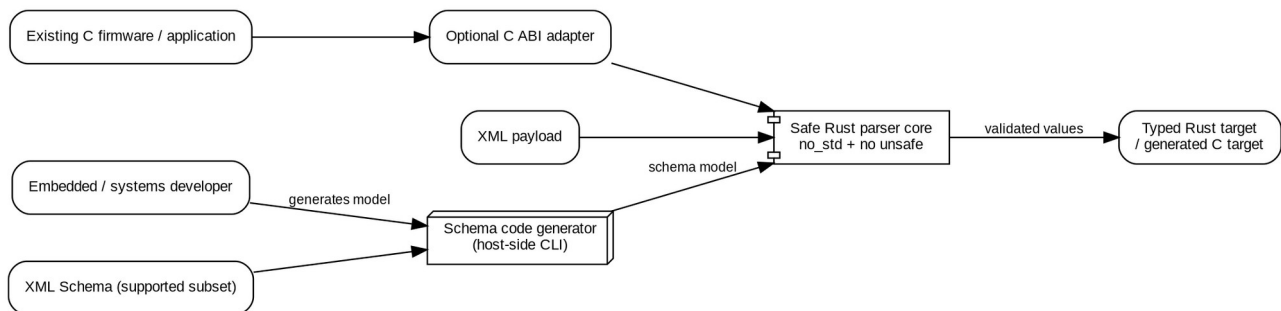


Figure 1. System context. The embedded runtime never parses XSD. Schema compilation is a host concern; runtime parsing receives generated, immutable schema descriptors.

4.3 Component architecture

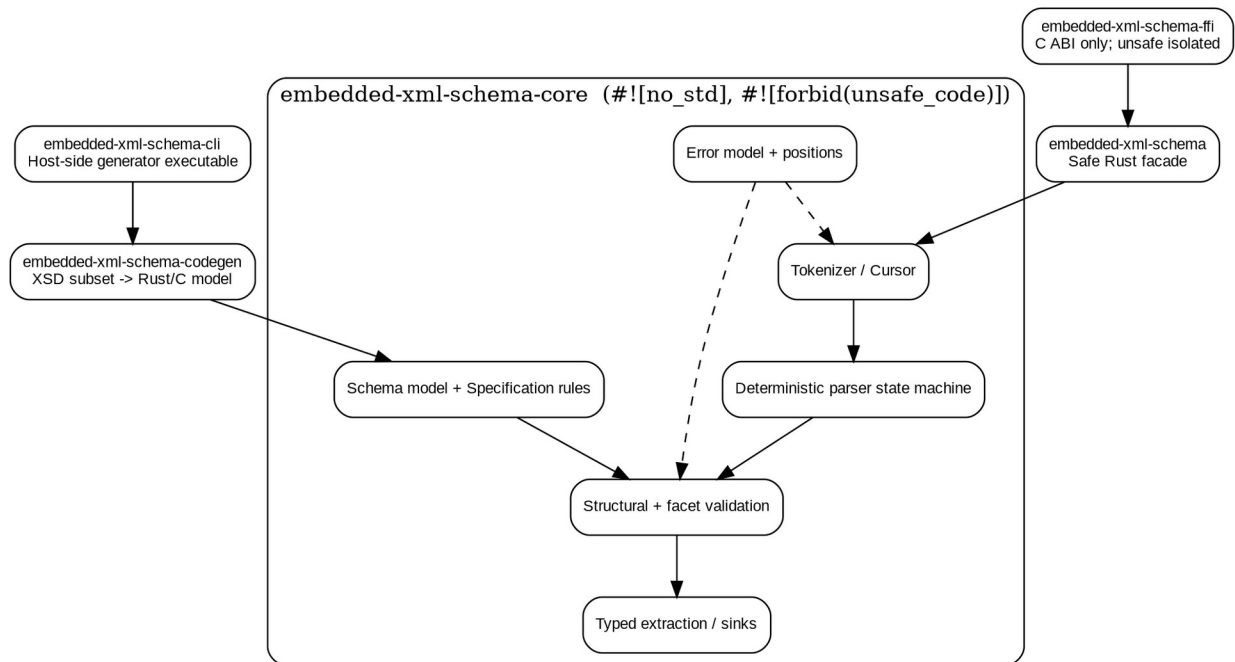


Figure 2. Component boundaries isolate unsafe interoperability from the safe no_std runtime.

4.4 Runtime parsing pipeline

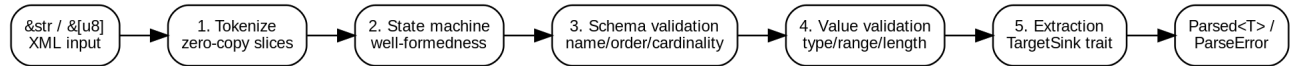


Figure 3. Parsing is modeled as a deterministic pipeline. Each stage can be unit-tested independently and exposes explicit errors instead of panics.

1. Input is wrapped by a bounds-safe cursor. The cursor is the only component that advances byte offsets.
2. Tokenizer emits lightweight events/slices without constructing a DOM.
3. The parser state machine enforces XML well-formedness and matching start/end tags.
4. Schema validation compares events against the current generated SchemaNode and cardinality state.
5. Facet validation converts and checks values before they cross into application targets.
6. TargetSink receives only validated data and constructs the typed result or calls an application hook.

4.5 Schema code-generation architecture

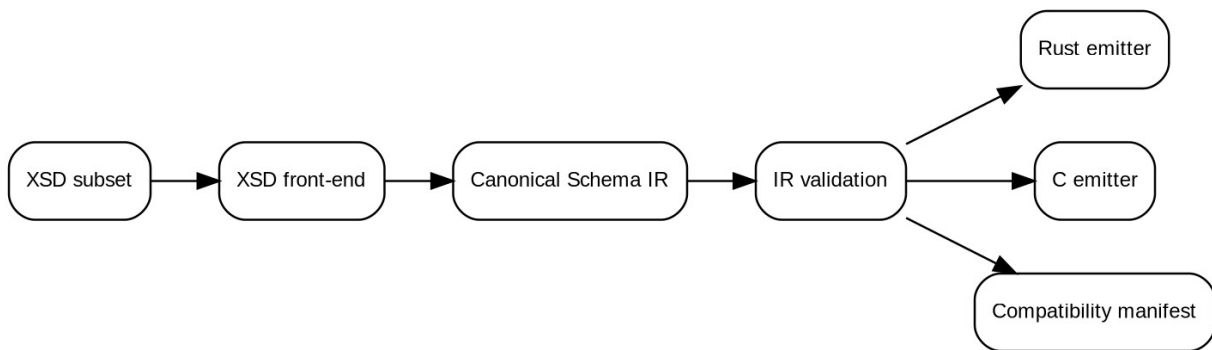


Figure 4. XSD is normalized into a canonical intermediate representation before code emission. This prevents the parser runtime, Rust emitter and C emitter from each implementing schema semantics independently.

Codegen layer	Responsibility
XSD front-end	Parse only the documented supported XSD subset; reject unsupported constructs with precise diagnostics.
Schema IR	Language-neutral canonical representation of elements, attributes, types, occurrence bounds and facets.
IR verifier	Ensures names, references, bounds and layout assumptions are internally consistent.
Rust emitter	Generates Rust target types, static Schema descriptors, conversion glue and optional borrowed lifetimes.
C emitter	Generates compatibility structures and descriptors with fixed-width types.
Manifest emitter	Writes generator/runtime schema ABI version, feature requirements and schema fingerprint.

4.6 Memory and ownership model

Concern	Decision
XML input	Borrowed for duration of parse/result lifetime.

Concern	Decision
Schema	Static generated data in the normal embedded profile.
Text values	Borrowed &str when no decoding is required; fixed/caller-owned target for materialized text; owned String only with alloc.
Repeated fields	Fixed-capacity target, callback sink, or Vec under alloc.
Parser state	Stack-resident bounded state; no hidden globals.
Callbacks	Borrowed trait/callback reference; no global callback registry.
FFI memory	Caller ownership is explicit. Rust does not free caller memory; Rust-owned opaque objects have matching destroy functions.

4.7 C ABI boundary

The FFI crate is optional and is intentionally not a mirror of internal Rust types. Its job is migration, not to freeze the internal architecture. All raw pointers are converted to checked slices/options at the boundary and only safe values enter the core.

Optional C ABI - isolated in embedded-xml-schema-ffi

```
// Conceptual ABI only
#[no_mangle]
pub unsafe extern "C" fn exs_parse(
    schema: *const exs_schema_t,
    input: *const u8,
    input_len: usize,
    target: *mut core::ffi::c_void,
    config: *const exs_config_t,
    error_out: *mut exs_error_t,
) -> exs_status_t;
```

5. Design Patterns and Architectural Patterns

The project should use patterns because they simplify safety, testability or compatibility, not because a catalog exists. The table below deliberately considers a broad set of patterns and marks whether each belongs in this project.

Pattern	Decision	Application / rationale
Hexagonal / Ports & Adapters	USE	Core parser depends on ports such as TargetSink and Hook; FFI/codegen/application-specific adapters stay outside.
Clean Architecture dependency rule	USE	Dependencies point inward toward the safe core; host tooling and C ABI cannot leak into parser logic.
Facade	USE	Parser offers one small entry point hiding tokenizer/state/validator/extraction internals.
Builder	USE	ParserConfig::builder() constructs explicit resource limits and strictness flags.
Strategy	USE	TargetSink, text conversion policy and optional hook behavior can vary without changing the state machine.

Pattern	Decision	Application / rationale
State	USE	XML syntax is implemented as an explicit state machine rather than ad-hoc nested conditionals.
Composite	USE	Generated SchemaNode hierarchy represents nested XML structures.
Specification	USE	Facets, occurrence constraints and allowed child/attribute rules are composable validation specifications.
Adapter	USE	C ABI adapter, generated model adapter and optional std helpers translate external interfaces into safe core contracts.
Visitor	USE/CONDITIONAL	Useful for host-side Schema IR traversal and emitters; avoid mandatory runtime visitor allocation.
Observer / Callback	CONDITIONAL	Element completion hooks support streaming consumption. Feature-gated to preserve minimal footprint.
Factory	USE	Codegen maps Schema IR nodes to emitted Rust/C type models through explicit factories/functions.
Command	USE IN TOOLING	Codegen passes can be represented as deterministic commands; runtime does not need it.
Chain of Responsibility	CONDITIONAL	Validation stages may be composed as a static chain; avoid dynamic dispatch in core.
Interpreter	CONDITIONAL	Schema rules act like a tiny declarative validation language, but runtime should execute generated descriptors rather than a generic interpreter.
Flyweight	USE	Schema names/metadata are static and reused; parser events borrow input slices instead of duplicating text.
Iterator	USE	Tokenizer can expose a pull/event iterator internally, enabling isolated tests and simple composition.
Newtype	USE	ByteOffset, Depth, SchemaNodeId and bounded counts prevent unit confusion and illegal cross-use.
Typestate	USE SELECTIVELY	Generator/config builders can make invalid construction states unrepresentable; avoid overcomplicating parser call sites.
RAII	USE	Any temporary resource or alloc-backed target is released automatically; especially relevant in std/alloc and test harnesses.
Dependency Injection	USE STATIC	Inject schema, config, sinks and hooks through generics/references; no service locator.
Null Object	OPTIONAL	A no-op Hook can remove Option branching where it improves generated code without size cost.

Pattern	Decision	Application / rationale
Bridge	NOT NEEDED	No independent class hierarchies justify a Bridge; traits/adapters already solve the variation.
Decorator	NOT CORE	Could decorate a TargetSink for tracing, but avoid runtime wrappers in the minimal embedded profile.
Proxy	NOT NEEDED	No remote/lazy object access requirement.
Mediator	NOT NEEDED	Components interact through a deterministic pipeline; a mediator would add indirection.
Memento	NOT NEEDED V1	The parser is one-shot; resumable/rollback parsing is explicitly out of v1 scope.
Prototype	NOT NEEDED	Generated immutable schema descriptors do not require cloning prototypes.
Singleton	REJECT	Global mutable parser/schema state would harm reentrancy and testing.
Service Locator	REJECT	Hidden dependencies conflict with deterministic embedded design.
Active Object / Actor	REJECT CORE	Concurrency is application-level. Core parsing is synchronous and deterministic.
Repository	NOT APPLICABLE	No persistence abstraction belongs in an XML parser.
CQRS / Event Sourcing	NOT APPLICABLE	No domain command/read model or event persistence problem exists.

5.1 Rust-specific design rules

- Make invalid states unrepresentable where it reduces runtime checks without creating unusable APIs.
- Prefer enums over integer status codes inside Rust; translate to integers only at FFI.
- Prefer borrowed slices and lifetimes to ownership/copying when the source buffer outlives the result.
- Use exhaustive matches to make state transitions auditable.
- Keep generic abstractions static where code size remains acceptable; use trait objects only when feature-driven flexibility justifies them.
- Do not use unsafe as a performance shortcut. Performance claims require benchmarks before an unsafe exception is even considered.

6. Test Architecture

Non-negotiable quality gate

Unit tests must reach 100% coverage of production logic by themselves. Integration tests and N2N/E2E tests are additional confidence layers and may not be used to make the unit coverage number reach 100%.

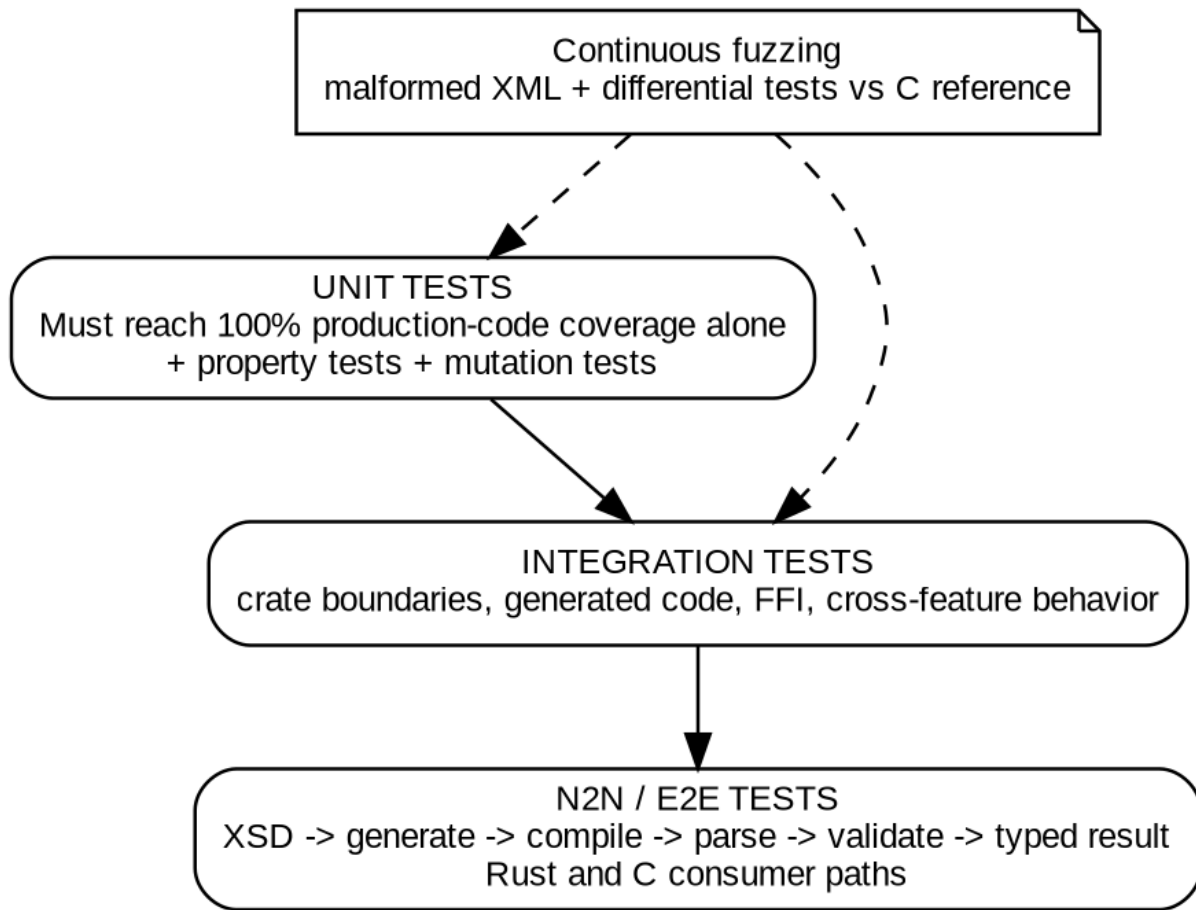


Figure 5. Test layers. Coverage reports are produced independently for the unit layer and the full workspace.

6.1 Unit-test layer - 100% standalone coverage

Every production branch of parser logic must have a direct unit-test path. Architecture is therefore decomposed into pure or deterministic functions rather than hiding behavior behind a single integration entry point.

Unit under test	Required coverage
Cursor / byte reader	Every boundary, EOF case, UTF-8 boundary and position update.
Tokenizer	Every token type, whitespace variation, malformed delimiter and truncated form.
Entity / character reference decoder	All predefined entities, numeric bases, invalid Unicode and overflow.
Name parser	Valid name classes, invalid starts, terminators and edge bytes.
State machine	Every legal transition plus every invalid transition.
Schema child matcher	Order modes, required/missing child, duplicate, min/max occurrence boundaries.
Attribute validator	Required/optional/unknown/duplicate attributes and conversion failures.
Value converters	Min/max values, overflow, underflow, signs, bool spellings, whitespace policy.

Unit under test	Required coverage
Facet validators	Every inclusive/exclusive bound and string length edge.
ResourceBudget	Exact limit, limit+1, zero, arithmetic overflow and repeated consumption.
TargetSink adapters	Success, capacity full, callback rejection and borrowed-value behavior.
Error positions	Byte/line/column correctness across LF and configured CRLF policy.
Schema IR verifier	Every supported rejection rule and every valid node form.
Rust/C emitters	Golden output fragments, escaping, identifiers and deterministic ordering.
FFI conversion helpers	Null, zero length, valid slices, invalid status mapping and error output.

Coverage command policy (illustrative):

```
# Unit coverage gate only - integration/E2E directories excluded from this run
cargo llvm-cov clean --workspace
cargo llvm-cov --workspace --lib \
  --all-features \
  --fail-under-lines 100 \
  --fail-under-functions 100 \
  --lcov --output-path target/coverage/unit.lcov

# Optional stronger gate where toolchain stability permits it:
# --branch --fail-under-regions 100
```

Generated output is tested through the generator logic and golden/compile tests. Generated files checked into fixtures should not artificially inflate or dilute runtime coverage; coverage inclusion/exclusion must be explicit and documented.

6.2 Property-based tests

- For every valid primitive value, serialize into XML text and verify parse(value) round-trips within the defined lexical format.
- Generated well-formed nested element trees within configured limits must never panic.
- For any input, parser offset must monotonically advance until success or error.
- Increasing a resource limit must not make a previously resource-valid document fail for a stricter resource reason.
- Equivalent entity/character references decode to the same scalar text value where supported.

6.3 Mutation testing

100% coverage is necessary but not sufficient. cargo-mutants (or equivalent) should run on core validation/conversion modules. Surviving mutants indicate weak assertions even if coverage is perfect. Critical modules should target zero surviving non-equivalent mutants before release.

6.4 Fuzzing and differential testing

Target	Strategy
Tokenizer fuzz	Arbitrary bytes/UTF-8; invariant: no panic, no out-of-bounds, always terminates under budget.
Parser fuzz	Arbitrary XML-like text plus generated schema; same invariants.

Target	Strategy
Structured fuzz	Generate near-valid XML and mutate delimiters, attributes, nesting and lengths.
Differential behavior	For the overlapping supported feature set, compare success/failure and extracted values against the original miniML-Parser C implementation.
Regression corpus	Every crash, hang, mismatch or security finding becomes a permanent test fixture.

6.5 Integration tests

Integration tests verify boundaries that unit tests intentionally isolate. They do not participate in the 100% unit coverage gate.

- Facade -> tokenizer -> state machine -> schema validator -> TargetSink using real generated descriptors.
- Core crate with each supported feature combination: default no_std, alloc, std, callbacks, cdata.
- Codegen Schema IR -> Rust emitter -> compile generated Rust fixture.
- Codegen Schema IR -> C emitter -> compile generated C fixture with a C compiler.
- FFI wrapper -> safe core using real pointers created by a controlled C/Rust harness.
- Version manifest mismatch and compatibility checks.
- Cross-crate error propagation preserving error code and source position.

6.6 N2N / E2E tests

N2N is treated here as the full end-to-end path from schema input to a consuming application. These tests should behave as a user would, invoking binaries/build steps rather than internal functions.

Scenario	End-to-end path
Rust happy path	food.xsd -> generator CLI -> generated Rust -> compile sample -> parse food.xml -> assert typed fields.
Rust rejection	schema -> generated Rust -> parse invalid XML -> assert stable public error code and position.
No-alloc embedded profile	schema -> generated fixed-capacity model -> cross-compile no_std sample -> parse host-equivalent fixture under the same capacities.
C consumer	XSD -> C artifacts -> build Rust staticlib + C sample -> call C ABI -> assert C structure values.
Compatibility differential	Same supported schema/XML -> original miniML C sample and Rust implementation -> compare result/value semantics.
Adversarial limits	Generate application -> feed too-deep/too-large/too-many-occurrences XML -> verify deterministic resource errors.

6.7 Test fixture taxonomy

Fixture group	Examples
valid/minimal	Single root, empty allowed content, optional fields absent.
valid/boundaries	Exactly min/max numeric values, exact occurrence limits, exact buffer capacities.

Fixture group	Examples
invalid/syntax	Truncation at every token position, mismatched end tags, bad quotes, illegal references.
invalid/schema	Unknown child, wrong order, missing required child, too many occurrences.
invalid/value	Overflow, malformed bool/number, facet violation.
security	Deep nesting, huge attribute list, billion-laugh-style constructs rejected, oversized numeric references.
compatibility	Cases shared with the original miniML examples and behavioral surface.
generated	Schemas exercising each supported XSD construct independently and in combinations.

7. Repository and Module Structure

```

embedded-xml-schema/
├── Cargo.toml                # workspace
├── README.md
├── LICENSE
├── SECURITY.md
├── CONTRIBUTING.md
├── docs/
│   ├── architecture/
│   ├── adr/
│   ├── compatibility/
│   └── supported-xsd.md
├── crates/
│   ├── core/                # no_std, forbid(unsafe_code)
│   │   └── src/
│   │       ├── cursor.rs
│   │       ├── tokenizer.rs
│   │       ├── state.rs
│   │       ├── schema.rs
│   │       ├── validate.rs
│   │       ├── convert.rs
│   │       ├── sink.rs
│   │       ├── budget.rs
│   │       └── error.rs
│   ├── facade/              # public ergonomic Rust API
│   ├── schema-ir/           # host-side canonical schema model
│   ├── codegen/             # XSD subset -> IR -> emitters
│   ├── cli/                 # generator executable
│   └── ffi/                 # isolated unsafe C ABI
├── fuzz/
│   ├── fuzz_targets/
│   └── corpus/
├── tests/
│   ├── integration/
│   ├── e2e/
│   ├── c-consumer/
│   └── fixtures/
├── examples/
│   ├── bare-metal-like/
│   ├── rust-generated/
│   └── c-compatible/
└── .github/workflows/
    ├── ci.yml
    └── coverage.yml
    
```

```
└─ fuzz-smoke.yml
└─ release.yml
```

A smaller initial workspace is acceptable, but the architectural boundaries should remain. In particular, FFI must never be placed inside the safe parser core merely for convenience.

8. CI/CD and Quality Gates

Gate	Required result
Formatting	cargo fmt --check passes.
Lint	cargo clippy with warnings denied for workspace-supported targets.
Unit tests	All pass on stable; unit-only coverage = 100% production lines/functions.
Mutation	Critical modules have no unexplained surviving mutants.
Integration	All crate-boundary and feature-combination tests pass.
N2N/E2E	Rust and C consumer pipelines pass.
no_std build	Core/facade supported no_std configurations compile.
Cross compilation	At least representative embedded targets compile, e.g. thumbv7em-none-eabihf and riscv32imac-unknown-none-elf where feasible.
MSRV	Workspace compiles on declared MSRV.
Miri	Host-testable safe APIs pass Miri where supported.
Fuzz smoke	Short CI fuzz run has zero crashes; longer fuzzing may run scheduled.
Dependency audit	cargo deny/audit policy passes; runtime dependency count is reviewed.
Docs	cargo doc with warnings denied; examples compile.
Generated determinism	Regeneration produces no unexpected diff.

Recommended pipeline

```
pull_request:
  fmt -> clippy -> unit-100%-coverage -> mutation-smoke
  -> integration -> e2e-rust -> e2e-c
  -> no_std-cross-build -> msrv -> docs

nightly/scheduled:
  extended-fuzz -> full-mutation -> benchmark-regression
```

9. Implementation Roadmap

Phase	Deliverable
Phase 0 - Bootstrap	Workspace, CI, lint policy, coverage gate, ADR template, no_std skeleton.
Phase 1 - Cursor + tokenizer	UTF-8-safe cursor, positions, XML names, text, attributes, entities, comments; 100% unit coverage before proceeding.
Phase 2 - Syntax state machine	Start/end tags, nesting, self-closing tags, strict progress/resource invariants.
Phase 3 - Schema runtime	Generated/static SchemaNode model, order and cardinality validation.
Phase 4 - Typed extraction	Primitive converters, facets, TargetSink, fixed-capacity patterns, borrowed strings.
Phase 5 - XSD codegen	Supported XSD frontend -> canonical IR -> deterministic Rust emitter.
Phase 6 - Integration + N2N	Generated Rust sample applications and full schema-to-value path.
Phase 7 - C compatibility	C emitter + optional FFI crate + cross-language N2N suite.
Phase 8 - Hardening	Differential testing vs miniML C, fuzzing, mutation testing, benchmarks, security documentation.
Phase 9 - Release	Semver/API review, MSRV lock, compatibility matrix, crates.io publication and release artifacts.

9.1 First implementation slice

Recommended first vertical slice

Implement a root element with one required integer child and one optional string attribute. Drive it through Cursor -> Tokenizer -> State -> Schema validation -> typed TargetSink, with 100% unit coverage. Only after that works should the XSD generator be introduced.

10. Risks, Constraints and Security Considerations

Risk	Mitigation
Scope explosion into full XML/XSD	Publish a precise supported-profile document and reject unsupported features explicitly.
Code size grows beyond embedded value	Track size benchmarks per release; keep features opt-in; avoid heavy runtime dependencies.
100% coverage becomes metric gaming	Add mutation testing and property/fuzz tests; coverage is a floor, not proof of correctness.
Generated code becomes complex/unsafe	Generated runtime descriptors must use only safe public constructors or audited const structures.

Risk	Mitigation
FFI undermines safety claim	State clearly: core is memory-safe Rust; FFI is an isolated unsafe compatibility boundary with separate tests.
Recursive schema/XML exhausts stack	Use explicit bounded state or reject beyond configured depth before recursion.
Zero-copy lifetimes make API difficult	Offer generated borrowed model for no-alloc and optional owned model behind alloc.
Differential reference contains bugs	Treat C reference as compatibility oracle, not correctness oracle; standards/profile requirements win on known discrepancies.
XSD parser itself becomes a second large project	Host codegen supports only a narrow subset; its parser may use established host-side Rust XML libraries because target footprint is irrelevant there.
Performance optimization pressures unsafe	Require benchmark evidence and ADR approval; default answer remains safe code.

11. Definition of Done / Acceptance Checklist

Area	Done when...
Core safety	core builds with <code>no_std</code> and <code>forbid(unsafe_code)</code> ; unsafe exists only in explicitly approved boundary crates.
API	Public Parser/Config/Error/Schema contracts are documented and semver-reviewed.
XML profile	All supported and unsupported XML features are documented with tests.
Schema profile	Supported XSD subset is documented with generator diagnostics for unsupported features.
Unit quality	Unit tests alone report 100% production line/function coverage.
Assertion quality	Mutation test baseline established; critical parser/validation modules have no unexplained survivors.
Integration	Feature combinations, generator/runtime boundary and FFI integration pass.
N2N/E2E	At least Rust and C schema-to-typed-output journeys pass from clean checkout.
Fuzzing	Core fuzz targets exist; crash corpus empty at release; regressions permanently captured.
Determinism	Generator output and parser behavior are deterministic.
Embedded	Representative <code>no_std</code> cross-target builds pass.
Security	SECURITY.md documents threat model, unsupported entity behavior and resource limits.
Docs	README has quick start, scope, comparison, generated example and migration guidance.

Area	Done when...
Release	MSRV, versioning, license and compatibility policy are fixed for the release.

12. Source Baseline and References

The architecture in this document is a proposed Rust design. The following source was used only to establish the behavioral baseline and original project capabilities:

miniML-Parser by kiishor - <https://github.com/kiishor/miniML-Parser>

- Original project description: simple/tiny validating XML parser in C for embedded applications.
- Original README states a 1.8 kB parser code footprint in one IAR ARM release build.
- Original parser validates against an `xs_element_t` schema tree, extracts typed content, supports callbacks and static/dynamic/relative target addressing.
- Original repository includes an XML schema code generator that emits C source/header structures.

Design note: exact compatibility with every miniML-Parser behavior is not assumed. The Rust implementation should document its compatibility surface and intentionally prefer memory safety, deterministic limits and a clear supported XML/XSD profile when behaviors conflict.

Appendix A - Initial Architecture Decision Records

ADR	Decision	Status	Rationale
ADR-001	Core is <code>no_std</code> and forbids unsafe code	Accepted	This is the primary product safety boundary.
ADR-002	XSD processing is host-side only	Accepted	Avoids embedding a large schema parser in firmware.
ADR-003	Canonical Schema IR separates XSD parsing from emitters	Accepted	Prevents duplicated semantics and enables Rust/C generation.
ADR-004	No DOM in v1 runtime	Accepted	Reduces allocation and memory footprint; streaming/event pipeline is enough for typed extraction.
ADR-005	UTF-8 only in v1	Accepted	Avoids encoding conversion complexity in embedded runtime.
ADR-006	DTD and external entities rejected	Accepted	Security and footprint simplification.
ADR-007	C ABI lives in separate crate	Accepted	Contains all raw-pointer unsafe boundary logic.
ADR-008	100% unit coverage is independent gate	Accepted	Ensures every production logic path has a direct unit-level test.
ADR-009	Resource limits are explicit public configuration	Accepted	Prevents resource-exhaustion behavior from being implicit.
ADR-010	Unsafe optimizations require benchmark + ADR	Accepted	Safety is default; exceptions require quantified evidence.

Appendix B - Suggested public milestones

Milestone	Public promise
0.1.0	Tokenizer + validating parser for minimal generated static schemas; no_std; safe core.
0.2.0	Primitive typed extraction, facets, resource limits, richer diagnostics.
0.3.0	XSD subset generator producing Rust models and schema descriptors.
0.4.0	Optional alloc conveniences and callbacks.
0.5.0	C artifact generation + C ABI preview.
0.9.0	Compatibility/fuzz/mutation hardening and API freeze candidate.
1.0.0	Stable supported XML/XSD profile, documented C ABI policy, full quality gates.